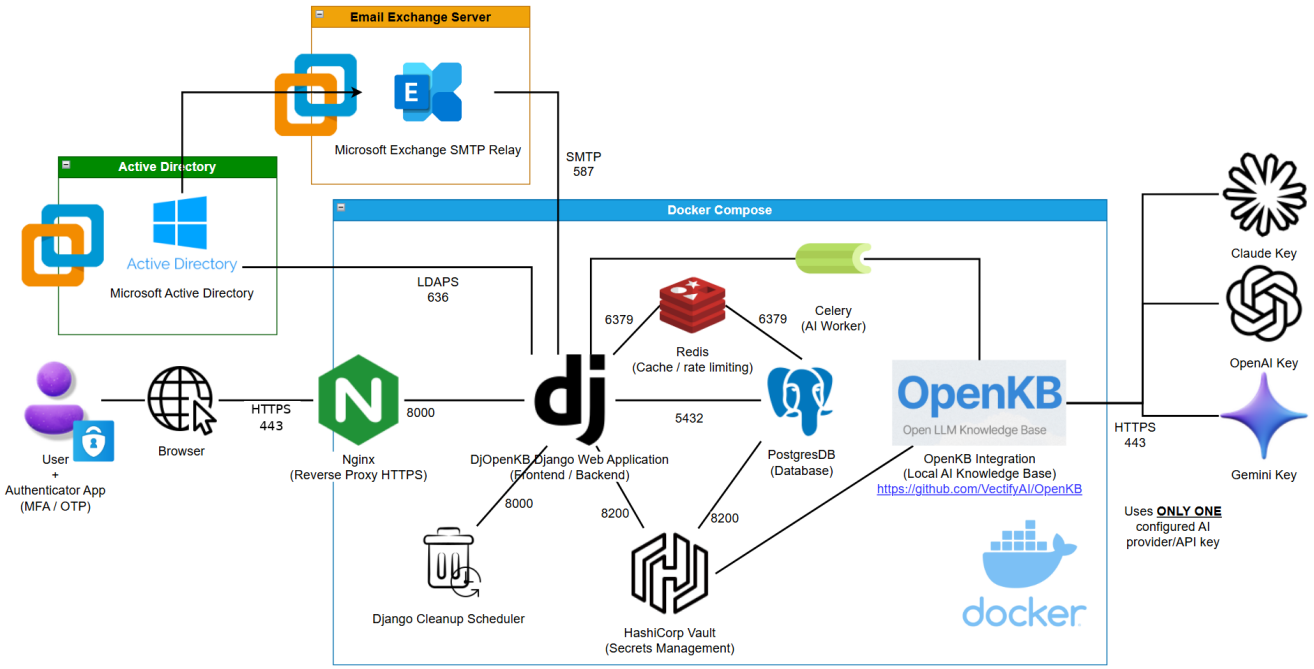
DjOpenKB

DjOpenKB is a Docker-based internal IT knowledge base built with Django. It provides a secure article website for IT documentation, public/internal article separation, user article suggestions, role-based review workflows, Active Directory / LDAPS login, local Django login, MFA, Vault-backed secrets, PostgreSQL, Nginx HTTPS, activity logging, and an integrated OpenKB AI chatbot.

The project is designed for a local VM, lab, or intranet-style deployment. A paid public domain is not required during development: users on the reachable internal network can use the browser-facing server IP over standard HTTPS port 443, for example `https://<INTERNAL_SERVER_IP>`. Replace `<INTERNAL_SERVER_IP>` with the approved internal address for the deployment. `localhost` and `127.0.0.1` refer only to the Linux server itself and are not remote-user addresses. When a firewall and final DNS name are later introduced, publish only HTTPS and update the trusted host/origin settings to the exact public address.

For a fresh installation, follow Deployment Guide. For later code, dependency, `.env`, or Vault secret changes, follow Update and Maintenance Guide. For a central map of `.env`, Vault secrets, Django Site settings, Nginx, Docker Compose, and restart requirements, use Configuration Reference. Optional SMTP relay setup, certificate preparation, workflow notifications, and authentication-lockout alerts are covered in SMTP Relay Setup and Notifications.

System Architecture



Main Features

- Internal IT wiki / knowledge base website.
- Login-protected public article browsing, title/keyword search, per-browser-session recent-search history, view tracking, voting, trending articles, most-liked articles, and most-recent articles. The main search dropdown shows up to five recent searches while empty and switches to live accessible article suggestions when text is entered.
- Separate internal article area for users with internal article access.
- Public/internal article visibility model with separate public and internal writer, approver, and manager roles.
- User article suggestion workflow with approval and pending-update review for published article edits.
- Optional SMTP relay workflow and security notifications: newly submitted/re-submitted public or internal review items notify matching reviewer groups in one Bcc-only message,

approval/Pending-failed decisions notify the current eligible article owner directly, and a recognised account reaching a new password/MFA lockout notifies eligible `Admin Users` in one Bcc-only alert. Internal messages omit internal titles, content, and review comments.
- Draft, pending approval, pending failed, published, and deletion-queued article states.
- Published article update workflow where user edits are held as pending updates while the current published version remains visible.
- Separate public and internal pending-review queues, including internal-only pending management for internal approvers/managers.
- Article owners can manage their own drafts, failed submissions, and pending updates within the visibility scope they are allowed to create.
- Published article deletion requires MFA confirmation. By default, a published article is hidden and placed in a recoverable admin deletion queue for 7 days; administrators can restore or permanently purge it. The retention setting can be set to `0` for immediate permanent deletion. Draft, pending, and pending-failed articles delete immediately.
- Dislike counts are limited to Article Manager, Internal Article Manager, and Admin Users. Normal users and approvers only see helpful/like counts.
- Admin tools for bulk article backup import/export, split exports, stray upload cleanup, orphan article management, published-article deletion queue recovery/purge, group/user role management, and site settings.
- Local Django login support.
- Active Directory login over LDAPS for valid domain users returned by the configured AD search base and user filter.
- Clear separation between local users and AD users.
- MFA support using authenticator-app one-time passwords.
- Authentication and append-only activity logging for important user, article, deletion queue, profile email/password, admin, AI, and maintenance actions. Search history and language selection changes are intentionally not logged.
- Configurable log retention and admin log display settings.
- Admin-configurable progressive password/MFA lockout policy with reset actions for administrators.
- Vault integration for sensitive secrets such as Django, database, LDAP, field-encryption, AI, and SMTP relay credentials.
- PostgreSQL database through Docker Compose.
- Redis-backed production cache for authentication lockouts, AI rate limits, fixed 24-hour AI quotas, background AI jobs, and query concurrency controls.
- Nginx HTTPS reverse proxy on standard host port 443 for direct internal access. A perimeter firewall may later forward public TCP 443 to the same host port. Nginx applies per-IP POST rate limits to login, MFA, admin MFA, AI, upload, and bulk-import submissions.
- Persistent OpenKB AI chatbot integration using `OpenKB-main/`, `openkb-data/`, and `openkb-data-internal/`. Questions run as short-lived Celery jobs so they continue while a signed-in user moves between normal site pages.
- AI resource controls: prompt length limit, short-burst rate limit and cooldown, a per-user fixed 24-hour quota, worker/query concurrency limits, timeout controls, related article recommendations, role-scoped AI indexing, and privacy-safe activity metadata.
- Markdown article rendering with sanitization, plus controlled playable video links for supported YouTube, Vimeo, and direct HTTPS `.mp4`/`.webm`/`.ogg` sources. SharePoint/OneDrive direct-video links are checked for anonymous accessibility before acceptance so viewers are not sent into an external sign-in flow.
- Image upload restrictions for article content, including protected image serving based on article visibility and ownership/review access.
- Multilingual UI support through Django translation files.
- Docker cleanup scheduler for routine cleanup tasks. The Compose stack separates the proxy, application, Vault, and worker-egress networks; application services use an unprivileged UID/GID, read-only filesystems, temporary `tmpfs` storage, capability dropping, and process limits where supported.
- Manual keyword suggestion refresh that scans the current title/body against existing manually created article keywords only.
- Admin-configurable article count per page with safe limits from 5 to 100.
- Login page is the only normal public app entry point; normal app/admin paths return 404 to anonymous users. `/robots.txt` is publicly reachable only so cooperative crawlers can see `Disallow: /`; application responses also carry a no-index header.

User Types and Rights

DjOpenKB currently uses a login-only website model. The root URL displays the login page. Anonymous users should not be able to browse normal article, internal article, search, profile, admin, AI, or upload pages; protected paths return 404 instead of exposing normal application pages. `/robots.txt` is the intentional crawler-only exception and contains no application content.

Main role matrix

Legend: ✓ = included in the role; ✗ = not included. "Manage published" means direct edit/delete capability in that scope. Owner updates are handled separately through the pending-update workflow.

Role / group	Core role	Public view	Internal view	Create own public	Create own
Anonymous visitor	No site access	✗	✗	✗	✗
Disabled User	Blocked account	✗	✗	✗	✗
Regular User	Public reader	✓	✗	✗	✗
Article Writer	Public contributor	✓	✗	✓	✗
Article Approver	Public reviewer	✓	✗	✗	✗
Article Manager	Public manager	✓	✗	✓	✗
Internal User	Internal reader	✓	✓	✗	✗
Internal Article Writer	Internal contributor	✓	✓	✗	✓
Internal Article Approver	Internal reviewer	✓	✓	✗	✗
Internal Article Manager	Internal manager	✓	✓	✗	✓
Admin Users	Full administrator	✓	✓	✓	✓

Important role notes:

- Writers can edit their own drafts and returned submissions. Changes to their own published articles become pending updates; they do not directly overwrite the published version.
- Published deletion requires an MFA confirmation. Writers use the owner workflow; managers and Admin Users manage published content within their scope.
- Approvers can edit content only while it is in their scope's pending-review flow.
- Article Managers see public dislike counts; Internal Article Managers see internal dislike counts; Admin Users see both.
- Django Admin requires normal sign-in/MFA and the separate Admin MFA gate. An optional IPv4/IPv6 allowlist can be enabled dynamically from Site settings.

Role interaction rules

Rule	Result
New active local or AD user without an elevated role	Receives `Regular User` as the fallback public-viewer r
Public writer, approver, or manager assigned	`Regular User` is removed because public viewing is a
Internal role assigned	Internal access is additive and also includes public art
Public and internal roles combined	Permissions remain scope-specific. For example, a pub
Writer combined with matching approver	Permissions are additive; the user can approve their o
Public Article Manager assigned	Replaces the lower public `Article Writer` and `Artic`
Internal Article Manager assigned	Replaces `Internal User`, `Internal Article Writer`
Disabled User assigned	Highest precedence. Standard role/admin groups and

Group membership is the baseline permission model. Direct user permission checkboxes are add-on permissions only; removing a direct permission does not remove a permission inherited from a group, and direct permissions do not grant Django Admin access.

Django's built-in **Active** checkbox controls whether an account can sign in at all. `Disabled User` is different: it retains an auditable account record but blocks Knowledge Repository access and presents the disabled-account page to a signed-in account.

Local and AD users are identified from account-source metadata rather than email domain. A local user may therefore use an address such as `alice@openkb.local` without being treated as an AD account.

Security Overview

Area	Security controls
Authentication	Login-only site access, local login, AD/LDAPS login limited by the configured A
Anonymous access	Only the login/language/static support paths and intentionally public `/robots`
User separation	Local and AD users are separated by account source; AD-managed password/
MFA	Authenticator-app OTP, MFA setup, MFA verification, MFA reset, and sensitive
Authorization	Group-based roles, public/internal scope checks, enforced role precedence, a
Articles	Public/internal visibility, draft/pending/pending failed/published/deletion-queu
Search and listing	Public search returns public results; internal-capable users can receive public
Keywords	Suggested keywords are manually refreshed and only come from existing ma
Uploads	Image-only allowlist, file validation, generated filenames, protected serving, a
Markdown / video	Sanitized rendered HTML to reduce XSS risk; supported standalone video link
AI chatbot	Login-protected persistent chat widget, role-scoped public/internal indexes, e
Password/MFA lockout	Progressive lockout policy stored in Site settings, with configurable stages, re
Logging	Separate authentication logs, append-only general activity logs, and admin a
Secrets	Vault-backed Django/database/LDAP/field-encryption/AI secrets. The bind-mo
Network and crawler controls	Nginx HTTPS reverse proxy, POST-only edge rate limits, 3 MB ordinary reques
Operations	Cleanup commands, cleanup scheduler, deployment checks, `.dockerignore`,

Article Workflow Summary

DjOpenKB keeps approved article content stable while still allowing controlled user updates. The same workflow exists for public and internal articles, but each visibility scope has its own permissions and review access.

```
New public article by Article Writer:
Draft -> Pending -> Pending failed / Published

New internal article by Internal Article Writer:
Draft -> Pending -> Pending failed / Published

User edits an already published article:
Published article remains visible
Edited version is saved as a pending update
Scope approver/manager/admin approves -> pending update replaces the published article
Scope approver/manager/admin rejects -> published article stays unchanged and feedback is s
```

Roles are additive. A user with only a writer role cannot approve; a user intentionally assigned the matching approver or manager role can approve their own matching-scope submission. This is an intended design decision for the project.

Published Article Deletion and Recovery

```
Draft / Pending / Pending failed delete:
Permanent deletion immediately

Published article delete, retention setting > 0:
MFA confirmation -> Deletion queued -> hidden from normal lists, search, article detail, an
Admin can restore or permanently purge during the recovery period
Cleanup scheduler permanently deletes it after the configured period

Published article delete, retention setting = 0:
MFA confirmation -> Permanent deletion immediately
```

The default published-article recovery period is 7 days. It is controlled in Django Admin under **Site settings -> Article deletion queue retention (days)**. The admin-only **Article deletion queue** records queued articles and provides restore and permanent-purge actions.

Public and internal scopes are separated:

Area	Public article	Int
Listing page	`/home/`	`/ir`
Create page	`/suggest/`	`/ir`
Owner article list	`/profile/articles/`	`/ir`
Pending review	`/profile/admin/pending-articles/`	`/ir`
Search	Public search; internal-capable users may also see internal results from main search	Int
OpenKB storage	Public published files under `openkb-data/`	Int
AI scope	Normal users query public index only	Int

Draft, pending, pending failed, deletion-queued, and unapproved pending-update content is not exposed through normal article detail traversal. Owners can open their own workflow articles, full admins can open all eligible workflow items, and approvers/managers use the explicit review/edit flows for their scope.

SMTP Relay Workflow and Lockout Notifications

SMTP workflow and lockout notifications are optional and disabled by default. The Django web service resolves current eligible role-group recipients and sends direct SMTP email using a Vault-stored service account. Public review submissions notify Public Article Approver/Manager/Admin Users, while internal review submissions notify Internal Article Approver/Manager/Admin Users. Reviewer pools use one Bcc-only message. Article approval and Pending-failed outcomes notify only the current eligible article owner with an authenticated DjOpenKB link. A recognised account reaching a new temporary password, normal MFA, or Django Admin MFA lockout sends one Bcc-only alert to active eligible `Admin Users`; retries during the same block do not send more mail, and unknown usernames remain log-only to prevent inbox flooding. TLS certificate and hostname validation remain enabled. Private-CA or self-signed Exchange relays can use one mounted public PEM/CRT trust certificate configured through `SMTP_RELAY_CA_CERT_FILE`; private keys and PFX/P12 bundles are never used. Follow SMTP Relay Setup and Notifications for certificate preparation, configuration, and testing.

Project Folder Structure

```
DjOpenKB/
|-- djopenkb/                    # Django project configuration
|-- kb/                          # Main Django application logic
|-- website/                     # Templates and static frontend files
|-- locale/                      # Django translation files
|-- documentations/              # Project setup, deployment, and feature guides
|-- docker/                      # Docker helper scripts
|-- nginx/                       # Nginx HTTPS reverse proxy configuration and cert scripts
|-- vault/                       # Vault configuration, bootstrap files, and local Vault ru
|-- (Docker volume: redis_data)   # Persistent Redis data is held in a named Docker volume,
|-- ldap-certs/                  # Public LDAPS and optional Exchange SMTP trust certificat
|-- OpenKB-main/                 # OpenKB source code used by the AI chatbot
|-- openkb-data/                 # Public OpenKB workspace and public article AI data
|-- openkb-data-internal/        # Runtime-created internal OpenKB workspace for internal a
|-- postgres-data/               # PostgreSQL local persistent data folder
|-- staticfiles/                 # Django collected static files
|-- scripts/                     # Utility and testing scripts
|-- manage.py                    # Django management command entry point
|-- Dockerfile                   # Django web container build file
|-- Dockerfile.postgres-vault    # PostgreSQL image with Vault password support
|-- docker-compose.yml           # Main Docker Compose stack
|-- .env                         # Local runtime configuration; do not share deployment cop
```

```
|-- .env.example                # Example runtime configuration
`-- requirements.txt            # Python dependencies
```

Folder Overview

djopenkb/

Contains the main Django project configuration.

This folder controls project-level settings such as installed apps, middleware, database configuration, session settings, security settings, static files, Vault integration, LDAP/LDAPS integration, and root URL routing.

Main files:

```
djopenkb/settings.py
djopenkb/urls.py
djopenkb/asgi.py
djopenkb/celery.py              # Celery application for background OpenKB AI jobs
djopenkb/wsgi.py
```

---

kb/

Contains the main Django app for the knowledge base.

This is where most application features are implemented, including article management, article suggestions, approval workflow, authentication handling, MFA, admin tools, uploads, OpenKB AI integration, activity logging, and management commands.

Important areas:

```
kb/models.py                # Database models for articles, votes, logs, MFA, site setting
kb/backends.py              # Authentication backend, including AD/LDAP login handling
kb/admin.py                 # Django admin registration and admin display settings
kb/admin_security.py        # Django Admin MFA gate and admin-session protection
kb/middleware.py            # Login-only, timeout, disabled-user, cache, and no-index midd
kb/permissions.py           # Role, public/internal scope, and precedence checks
kb/urls.py                  # App-level URL routes
kb/views/                   # Main page, article, admin, auth, MFA, AI, and service views
kb/management/commands/     # Custom Django management commands
kb/migrations/              # Database migrations
kb/templatetags/            # Custom template filters/tags
```

General view grouping:

```
kb/views/main.py            # Article listing, search, detail pages, voting, and normal br
kb/views/suggestions.py     # User article suggestions, drafts, edits, and article image u
kb/views/admin.py           # Admin-only article review and maintenance tools
kb/views/auth.py            # Login/profile/account-related pages
kb/views/mfa.py             # MFA setup, verification, and reset views
kb/views/ai.py              # Queue/cancel endpoints for the persistent OpenKB AI widget
kb/views/ai_jobs.py         # Encrypted short-lived job records and secure status polling
kb/tasks.py                 # Celery task that runs one background OpenKB AI job
kb/views/security.py        # Plain-text /robots.txt endpoint
kb/views/services.py        # Shared helper logic and compatibility re-exports
kb/views/services_bulk.py   # Bulk import/export, ZIP splitting, and restore helpers
kb/views/services_search.py # Search ranking, related articles, and suggestions
kb/views/services_ai.py     # OpenKB AI sync/query, quotas, rate limiting, and recommendat
```

---

website/

Contains the frontend files used by the Django app.

```
website/templates/           # HTML templates
website/templates/admin/     # Django Admin template overrides
website/static/              # CSS, JavaScript, images, icons, and other static assets
```

The templates cover the base layout, login pages, article pages, profile pages, admin tools, MFA pages, AI chat UI, and Django Admin display improvements.

---

locale/

Contains Django translation files for the multilingual interface.

Each language has its own folder with `.po` and `.mo` files:

```
locale/<language>/LC_MESSAGES/django.po
locale/<language>/LC_MESSAGES/django.mo
```

After editing `.po` files, compile the `.mo` files:

```
docker compose exec web python manage.py compilemessages
```

---

documentations/

Contains the main documentation files for setup, testing, and feature explanation.

```
documentations/DEPLOYMENT_GUIDE.md
documentations/CONFIGURATION_REFERENCE.md
documentations/UPDATE_AND_MAINTENANCE_GUIDE.md
documentations/FULL_FEATURE_DOCUMENTATION.md
documentations/LDAP_LDAPS_SETUP.md
documentations/WINDOWS_SERVER_2022_AD_LDAPS_SETUP.md
documentations/PUBLIC_EXPOSURE_HARDENING.md
documentations/SMTP_RELAY_NOTIFICATIONS.md
```

Use the deployment guide for first-time Linux server setup, initial `createsuperuser` creation, and reboot persistence. Use the update and maintenance guide for Git/VS Code deployments, direct server edits, dependency changes, `.env` updates, and Vault secret updates. Use the LDAPS and Windows Server guides for Active Directory login, the public-exposure guide for edge hardening, and the SMTP guide for relay certificate setup, notifications, and testing.

---

nginx/

Contains the Nginx reverse proxy configuration.

Nginx provides HTTPS access to the Django web container on standard host port 443. A perimeter firewall can publish or forward only external TCP 443 to the same host port. Nginx uses a read-only root filesystem with a writable `/tmp` `tmpfs`; its temporary request paths are configured directly below `/tmp` so uploads and proxied requests continue to work without weakening the container filesystem.

Important paths:

```
nginx/nginx.conf
nginx/certs/
```

The `nginx/certs/` folder is used for local HTTPS certificates. Do not share private key files such as:

```
nginx/certs/localhost.key
```

---

vault/

Contains HashiCorp Vault configuration and local Vault runtime folders.

Vault is used to store sensitive values such as Django secret key, PostgreSQL password, field-encryption key, LDAP bind password, and AI API keys. The app token at `vault/keys/djopenkb-app-token.txt` is created with owner/group `0:10001` and mode `0440`, allowing only root and the unprivileged application group to read it.

Important paths:

```
vault/config/               # Vault configuration
vault/bootstrap/            # First-time bootstrap secret file and helper scripts
vault/keys/                 # Vault tokens and unseal keys, do not share
vault/file/                 # Vault runtime data, do not share
vault/logs/                 # Vault logs
vault/scripts/              # Vault setup/helper scripts
```

The bootstrap file is only for first-time setup:

`vault/bootstrap/djopenkb.env`

After Vault is seeded, do not share this file.

---

ldap-certs/

Contains the exported CA certificate used by Django to verify the Windows Server LDAPS certificate.

Expected file:

`ldap-certs/ad-ca.crt`

This folder is mounted into the web container so Django can validate LDAPS properly.

OpenKB-main/

Contains the OpenKB source code used by the AI chatbot integration.

The Django project calls OpenKB from this folder when answering AI chat queries and working with article data.

The `.env` setting normally points to this folder:

`OPENKB_BASE_DIR=OpenKB-main`

---

openkb-data/

Contains the public OpenKB workspace used by the chatbot for public article indexing.

This folder stores the public OpenKB data/index workspace and published public article files used by the AI integration. Internal article data is not stored here.

Before using the chatbot on a fresh server, initialise this folder once using the temporary host virtual-environment procedure in `documentations/DEPLOYMENT_GUIDE.md` Section 9. Run `openkb init` from `/opt/DjOpenKB/openkb-data`, use the same model as `OPENKB_AI_MODEL`, and leave the OpenKB API-key prompt blank because the production key is supplied through Vault. Do not re-run initialisation over a healthy existing workspace.

Then sync published Django articles into OpenKB. By default this rebuilds both the public index and the internal public+internal index:

`docker compose exec web python manage.py sync_openkb_ai`

If OpenKB is not initialized, the chatbot may fail or not detect the article data correctly.

---

openkb-data-internal/

Contains the separate internal OpenKB workspace used for internal article indexing.

Internal article Markdown and generated internal AI summaries are kept here instead of being placed into the public `openkb-data/` tree. This prevents the public OpenKB index from accidentally retrieving internal-only content.

The normal sync command rebuilds both public and internal indexes by default:

```
docker compose exec web python manage.py sync_openkb_ai
```

You can also sync a specific scope:

```
docker compose exec web python manage.py sync_openkb_ai --scope public
docker compose exec web python manage.py sync_openkb_ai --scope internal
docker compose exec web python manage.py sync_openkb_ai --scope all
```

To check that internal articles are not present in the public OpenKB tree:

```
docker compose exec web python manage.py check_internal_article_isolation --sync-first
```

---

postgres-data/

Stores PostgreSQL local persistent data when using a bind-mounted database folder.

Do not delete this folder unless you intentionally want to reset the local database.

---

staticfiles/

Contains static files collected by Django using:

```
docker compose exec web python manage.py collectstatic --noinput
```

Nginx serves these collected static files.

---

scripts/

Contains utility scripts used for testing or maintenance.

Example:

```
scripts/test_ldaps.sh
scripts/test_ldaps_tls.py
scripts/apply_admin_mfa_10min_default_patch.py
scripts/fix_admin_mfa_every_entry.py
```

These scripts help confirm whether the Django container can resolve and connect to the LDAPS server, or apply narrowly scoped administrative maintenance patches. Translation files are maintained through Django `makemessages` / `compilemessages` workflows.

OpenKB AI Chatbot Background Jobs, Quotas, and Production Controls

Ask OpenKB AI is available only to signed-in users. The browser submits a question to Django, which creates an opaque job ID and places the work on the dedicated `ai-worker` Celery queue. The AI query continues if the user moves between normal site pages in the same browser tab; the widget resumes polling and displays the finished response on the page currently open.

The widget stores only its open state, completed messages, pending job IDs, and unfinished draft text in the browser tab's `sessionStorage`. It is not saved as chat history in PostgreSQL. Selecting **Clear chat** removes the visible thread and asks the server to discard queued/running job results;

an already-running OpenKB subprocess is allowed to finish safely, but its result is not returned to the cleared chat.

Current production controls:

Control	Default behaviour
Short-burst limit	5 accepted questions within 60 seconds per signed-in account
Burst cooldown	30 minutes after exceeding the short-burst limit
Fixed per-user quota	20 accepted questions in a fixed 24-hour window; editable in **Django Admin → Site**
Quota window	The first accepted question starts the 24-hour timer. Later questions increase the co
Prompt length	1000 characters
OpenKB query timeout	90 seconds
Background worker	One `ai-worker` service with Celery concurrency 1 by default
Job lifetime	Encrypted temporary prompt/result record expires after 30 minutes by default
Polling	Browser checks pending job status every 2 seconds by default

The fixed 24-hour quota is held as one small Redis counter per user. It avoids per-prompt database writes and requires no scheduled reset task. The Admin setting accepts values from 1 to 1000 and defaults to 20.

The chatbot is role-scoped:

User access	AI data used
Public article access only	Public OpenKB index under `openkb-data/`
Internal article access	Internal OpenKB index under `openkb-data-internal/`, containing published publ

Question and answer text is Fernet-encrypted before it is placed in the temporary Redis-backed job record. Celery receives only the opaque job ID. The worker checks the owner account and current article scope before querying, and the polling endpoint checks ownership and scope again before returning a result. Long-lived activity logs record operational metadata such as question length, scope, quota usage, and outcome—not the question or answer text.

Keep `OPENKB_AI_WORKER_CONCURRENCY=1` for the current single-VM deployment unless resource capacity, provider limits, and OpenKB behaviour have been assessed for a higher setting. Redis remains required in production so job records, rate limits, quotas, and shared query controls work consistently across services.

Public-Exposure Hardening

Before the perimeter firewall permits public traffic, follow `documentations/PUBLIC_EXPOSURE_HARDENING.md`. This release adds Nginx edge throttling, smaller default request limits, private Docker backend networks, unprivileged Django/Celery containers, read-only filesystems with controlled `tmpfs` storage, a fixed maximum session lifetime (8 hours by default), Vault token group permissions, and configurable AD search-scope controls. During direct internal-IP access, `DJANGO_ALLOWED_HOSTS` uses the reachable server IP and `DJANGO_CSRF_TRUSTED_ORIGINS` uses the exact standard HTTPS origin, such as `https://<INTERNAL_SERVER_IP>`. After a firewall or DNS name is introduced, update both values to the exact public address seen by the browser.

Quick Deployment Summary

Full deployment steps are in:

`documentations/DEPLOYMENT_GUIDE.md`

Basic flow:

```
git clone https://github.com/ErinFlyingSkyRocket/DjOpenKB.git
cd DjOpenKB
cp .env.example .env
nano .env
sh vault/bootstrap/generate-secrets.sh
```

```
chmod +x nginx/certs/generate-localhost-cert.sh
./nginx/certs/generate-localhost-cert.sh <DIRECT_INTERNAL_SERVER_IP>

# Complete the one-time host OpenKB initialisation from Deployment Guide Section 9.
# The web container runs migrations, schema repair, and collectstatic automatically at star
docker compose up -d --build
docker compose exec web python manage.py createsuperuser
docker compose exec web python manage.py seed_djopenkb_roles --assign-missing-users
docker compose exec web python manage.py sync_openkb_ai --scope all
docker compose exec web python manage.py check_internal_article_isolation --sync-first
docker compose exec web python manage.py check --deploy
# After first login, review Site settings → Authentication lockout policy stages and OpenKB
```

Access the website using:

`https://<server-ip>`

For host-reboot persistence, enable the `djopenkb.service` systemd unit documented in `documentations/DEPLOYMENT_GUIDE.md`. The boot service starts existing Compose containers with `docker compose up -d`; normal code deployments still use `docker compose up -d --build`. The long-running Compose services also retain `restart: unless-stopped` policies for individual container recovery.

Updating Later

For future updates:

```
cd /path/to/DjOpenKB
git pull --ff-only
docker compose up -d --build
docker compose ps
docker compose logs --tail=100 web
docker compose logs --tail=100 ai-worker
docker compose exec web python manage.py check --deploy
```

If local files block `git pull`, check first:

```
git status
```

Then either commit your changes, stash them, or restore unwanted local changes before pulling.

Useful Maintenance Commands

Emergency Admin IP Allowlist Recovery

If the Django Admin IP allowlist is enabled with an incorrect IPv4/IPv6 address or CIDR range and you accidentally block your own management device, recover access from the Linux server shell:

```
cd /opt/DjOpenKB
sudo docker compose exec web python manage.py reset_admin_ip_allowlist
```

This command fully resets the dynamic Admin IP allowlist. It **disables the allowlist and permanently clears all configured IPv4/IPv6 addresses and CIDR ranges** and does **not bypass normal login, superuser access, normal MFA, or the separate Admin MFA gate**. After access is restored, configure a new allowlist from scratch in **Django Admin → Site settings → Django Admin access restrictions**, then enable the allowlist again.

No `.env` or Nginx change is required for this recovery.

Check container status:

```
docker compose ps
```

View logs:

```
docker compose logs --tail=100 web
docker compose logs --tail=100 ai-worker
docker compose logs --tail=100 nginx
docker compose logs --tail=100 vault
docker compose logs --tail=100 cleanup-scheduler
```

Run Django checks:

```
docker compose exec web python manage.py check
docker compose exec web python manage.py check --deploy
```

Sync OpenKB AI article data. By default this rebuilds both the public index and the internal public+internal index:

```
docker compose exec web python manage.py sync_openkb_ai
docker compose exec web python manage.py sync_openkb_ai --scope public
docker compose exec web python manage.py sync_openkb_ai --scope internal
```

Inspect or run cleanup commands:

```
# General and Django Admin activity logs
docker compose exec web python manage.py cleanup_activity_logs --dry-run
docker compose exec web python manage.py cleanup_activity_logs --noinput

# Authentication and MFA logs
docker compose exec web python manage.py cleanup_auth_activity_logs --dry-run
docker compose exec web python manage.py cleanup_auth_activity_logs --noinput

# Published-article deletion queue
docker compose exec web python manage.py cleanup_article_deletion_queue --dry-run
docker compose exec web python manage.py cleanup_article_deletion_queue --noinput
```

Verify crawler controls after deployment:

```
curl -k https://<server-ip>/robots.txt
curl -k -I https://<server-ip>/login/
```

Expected results are `User-agent: *` plus `Disallow: /` from `/robots.txt`, and an `X-Robots-Tag: noindex, nofollow, noarchive, nosnippet, noimageindex` header on the login response.

Compile translations after editing `.po` files:

```
docker compose exec web python manage.py compilemessages
```

Development vs Production Runtime Notes

During active development, the Docker Compose bind mount below is useful because code edits on the host appear inside the container immediately:

```
- .:/app
```

For a production or final demonstration deployment, remove the full project bind mount where possible and rebuild the image instead. This reduces the chance that host-side files such as `.env`, Vault material, local certificates, Git metadata, or temporary files are visible inside the running web container.

Production deployments should keep `REDIS_URL=redis://redis:6379/1` and `DJANGO_ALLOW_LOCAL_CACHE_FA` so authentication lockout, AI rate limits, and AI concurrency controls are shared across all Gunicorn workers.

A `.dockerignore` file should remain in place so local secrets and runtime data are not copied into the Docker image during `COPY . /app/`.

Backup and Restore Notes

For a real intranet or production-style deployment, back up these areas regularly:

```
PostgreSQL database
Vault data and recovery material
Redis data if persistence is enabled
openkb-data/
openkb-data-internal/
article uploaded images
generated OpenKB Markdown files
.env and deployment configuration, stored securely
```

Do not only back up the source code. The running system also depends on local database state, Vault state, uploaded files, and OpenKB data.

Recommended operational practice:

```
1. Create backups regularly.
2. Store backups securely.
3. Test restore steps before relying on them.
4. Do not store Vault keys, private keys, or database backups in public Git repositories.
```

Bulk article export/import backup

Use the admin **Bulk import/export articles** tool when you need an application-level article backup or migration package. The export includes article body content, titles, keywords, published status, pending-update data, review notes/history where applicable, and referenced image files.

The import upload limit is 100 MB per ZIP file. Split exports are packaged so that each inner part ZIP can be imported one by one instead of uploading one very large file.

Files Not to Share

Do not commit or share these files/folders:

```
.env
.env.*
!.env.example
vault/bootstrap/djopenkb.env
vault/keys/*
vault/file/*
openkb-data/.openkb/
openkb-data-internal/.openkb/
.openkb-venv/
ldap-certs/
nginx/certs/*.key
postgres-data/
exported article ZIP backups
```

These may contain secrets, tokens, private keys, database content, or local runtime state.

Final Security Reminder

Before deploying outside a local lab, confirm:

```
DEBUG=False
DJANGO_ALLOWED_HOSTS and DJANGO_CSRF_TRUSTED_ORIGINS match the exact browser URL
HTTPS is working through Nginx and the certificate covers the direct IP or final DNS name
LDAPS certificate validation is working and LDAP_USER_SEARCH_BASE / LDAP_USER_FILTER are re
Vault is initialized and sealed/unsealed correctly
vault/keys/djopenkb-app-token.txt shows owner/group 0:10001 and mode 440
app-permissions-init exits successfully before application services start
Django check --deploy has been reviewed
Backups and restore steps are documented
```

Redis-backed rate limiting/cache is enabled for production
Nginx login/MFA/AI/upload/import rate-limit behaviour has been tested
OpenKB AI rate limiting, concurrency control, and activity logging are enabled
Admin-configurable password/MFA lockout stages have been reviewed
Published article deletion queue retention has been reviewed (`7` default; `0` means immedi
`/robots.txt` and `X-Robots-Tag` behaviour have been verified